技术部第四次技术分享会

基础数据结构与 Java 容器

姚雨森　技术部分管主席

回顾数组：长度需要提前确定

数组把一组同类型的数据放在一起，通过从 0 开始的下标访问。

```
int[] prices = new int[3];
prices[0] = 12;
prices[1] = 8;
prices[2] = 25;
System.out.println(prices.length); // 3
```

- 数组一旦创建，长度就固定了；可以修改元素，不能直接增加位置。
- 预留太少，后续数据放不下；预留太多，又有空余位置需要管理。
- 商品不断增加或下架时，我们希望能直接添加、删除元素，让保存的数据数量随需求变化。

除了位置编号，我们还需要数据的标签

数组用下标表示**位置**，但我们有时更想用一个**标签**找到数据。

- 按位置访问：第 0 件商品的价格是多少？
- 按标签查找：商品编号 P1001 对应的价格是多少？
- 这种用于查找的标签称为键（key），对应的数据称为值（value）。

数组也能配合查找代码完成任务，但我们希望有更直接的存储方式：

- **数量可变的列表**：方便添加、删除元素，不必自己安排数组长度。
- **键与值的映射**：根据商品编号等标签，保存和查询对应的数据。

ArrayList：可以增减元素的列表

可以把 ArrayList 理解为可以变化长度的数组，直接添加或删除元素。

```
// import java.util.ArrayList;
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
System.out.println(products.size()); // 2
System.out.println(products.get(0)); // Apple
products.set(0, "Bread");
products.remove(1);
System.out.println(products.size()); // 1
```

add 添加，get 读取，set 修改，remove 删除。

下标仍从 0 开始；元素数量用 size() 获取。

get、set、add：对应数组上的哪些操作

数组：用下标读取和赋值

```
String[] products = {"Apple"};

String first = products[0];

products[0] = "Bread";
```

ArrayList：调用对应的方法

```
ArrayList<String> products =
    new ArrayList<>();
products.add("Apple");
String first = products.get(0);
products.set(0, "Bread");
products.add("Milk");
```

- `get(i)` 对应数组的 `products[i]`：读取已有位置。
- `set(i, value)` 对应数组的下标赋值：替换元素，数量不变。
- `add(value)` 在末尾新增元素，数量加一；数组没有直接追加的操作。

空列表要先用 `add` 添加元素，不能用 `set(0, ...)` 创建第一个元素。

尖括号指定列表中的元素类型

ArrayList<String> 表示这个列表用来存放 String 类型的元素。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
String first = products.get(0);
// products.add(12); // Type mismatch
```

- 尖括号中的类型叫类型参数，这种写法称为泛型。
- 必须填写**引用类型**，例如 String、Student 等类。
- int、double 等**基本类型**不能直接填进去。

```
// ArrayList<int> prices = new ArrayList<>(); // Invalid
```

那么，要保存一组整数，应该用什么类型?

存放整数时，使用 Integer

保存 int 数据时，写成 ArrayList<Integer>；保存 double 数据时，使用 ArrayList<Double>。

```
ArrayList<Integer> prices = new ArrayList<>();
prices.add(12);
int price = prices.get(0);
```

基本类型	对应的类	基本类型	对应的类
byte	Byte	float	Float
short	Short	double	Double
int	Integer	char	Character
long	Long	boolean	Boolean

这些对应关系涉及**包装类**，这里先记住用法。

按下标删除：后面的元素会向前补位

`remove(index)` 删除指定位置的元素，列表的元素数量减一。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
products.add("Bread");
products.remove(1);
System.out.println(products);         // [Apple, Bread]
System.out.println(products.get(1)); // Bread
System.out.println(products.size()); // 2
```

- 删除前，Milk 的下标是 1，Bread 的下标是 2。
- 删除后，Bread 的下标变为 1，不会留下空位。

按值删除：只删除第一个匹配的元素

知道商品名称时，可以直接使用 remove(value)。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
products.add("Apple");
boolean removed = products.remove("Apple");
System.out.println(removed);  // true
System.out.println(products); // [Milk, Apple]
boolean found = products.remove("Tea");
System.out.println(found);    // false
```

- 有重复元素时，只删除从前往后找到的第一个。
- 找到并删除返回 true；没有找到返回 false，列表不变。

整数列表：删除下标，还是删除数值

对 ArrayList<Integer>，直接传入整数会被当作下标。

```
ArrayList<Integer> prices = new ArrayList<>();
prices.add(12);
prices.add(8);
prices.add(1);
prices.remove(1);
System.out.println(prices); // [12, 1]
prices.remove(Integer.valueOf(1));
System.out.println(prices); // [12]
```

- remove(1)：删除下标为 1 的元素，本例中是 8。
- remove(Integer.valueOf(1))：删除数值为 1 的元素。

先记住这两种写法，不需要展开包装类的转换细节。

查找元素：是否存在，第一次出现在哪里

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
products.add("Apple");
System.out.println(products.contains("Milk")); // true
System.out.println(products.contains("Tea"));  // false
System.out.println(products.indexOf("Apple")); // 0
System.out.println(products.indexOf("Tea"));   // -1
```

- `contains(value)`：只想知道元素是否存在。
- `indexOf(value)`：需要第一个匹配元素的下标。
- `-1` 表示没有找到，不能把它作为下标传给 `get`。

通过下标遍历列表

需要同时使用**位置和元素**时，用下标逐个读取。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
for (int i = 0; i < products.size(); i++) {
    System.out.println(i + ": " + products.get(i));
}
// 0: Apple
// 1: Milk
```

- 数组中的 `length`，在这里换成 `size()`。
- 数组中的 `products[i]`，在这里换成 `products.get(i)`。
- 循环条件是 `i < products.size()`，不能写成小于等于。

增强 for：直接取得每一个元素

只需要**元素本身**时，增强 for 可以省去下标操作。

```
ArrayList<String> products = new ArrayList<>();
products.add("Apple");
products.add("Milk");
for (String product : products) {
    System.out.println(product);
}
// Apple
// Milk
```

- 冒号右边是要遍历的列表 products。
- 冒号左边声明变量 product，每轮取得一个元素。
- 按列表顺序访问元素；同样的写法也适用于数组。

增强 for 中，不要直接增删原列表

遍历过程中直接增删原列表，可能抛出 ConcurrentModificationException。

```
// Incorrect: do not modify the list this way.
for (String product : products) {
    if (product.equals("Milk")) {
        products.remove(product);
    }
}
```

- 没有抛出异常，也不能说明这种写法可靠。
- 如果只需删除一个已知值，可以直接调用 products.remove("Milk")，不必遍历。
- 需要筛选多个元素时，可以先把要保留的元素放入**另一个列表**。